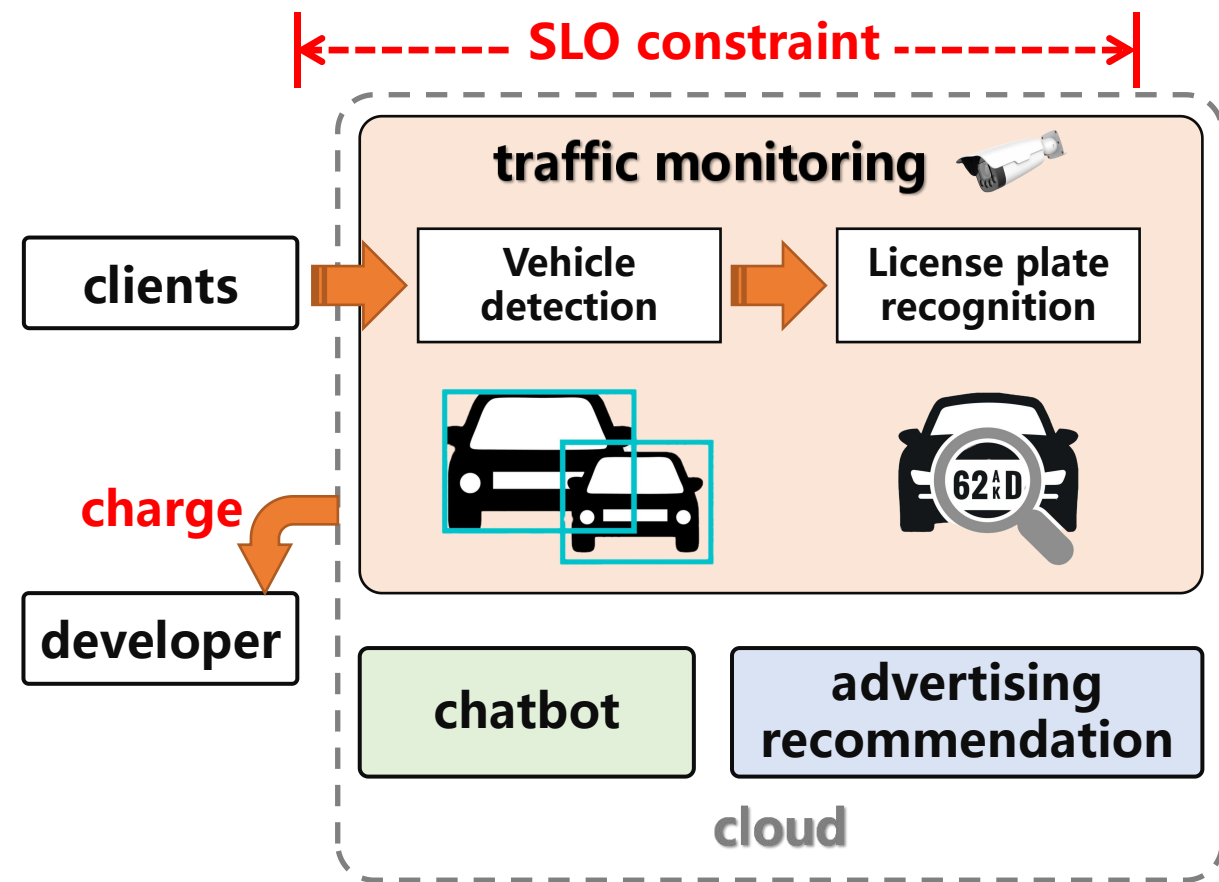


SLOpt: Resource Scaling for DNN Workload with Tail Latency Constraint

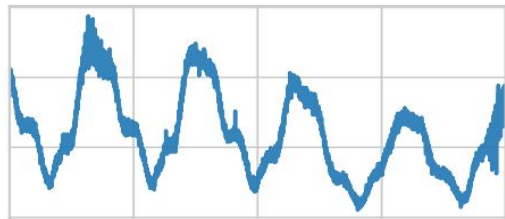
Zhixin Zhao, Yitao Hu, Guotao Yang, Yingqin Chen, Wenxin Li, Wenyu Qu

May 2023

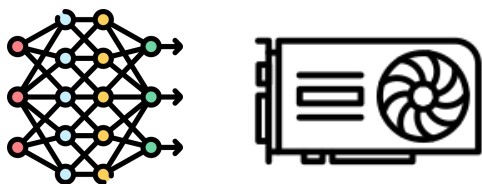
- DNN-based applications
- Inference serving system
 - latency objective
 - serving cost



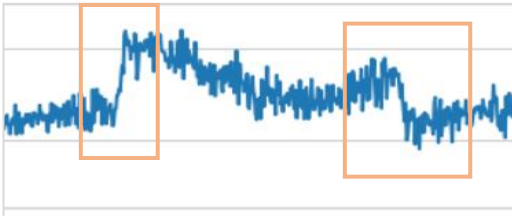
Challenge for inference system



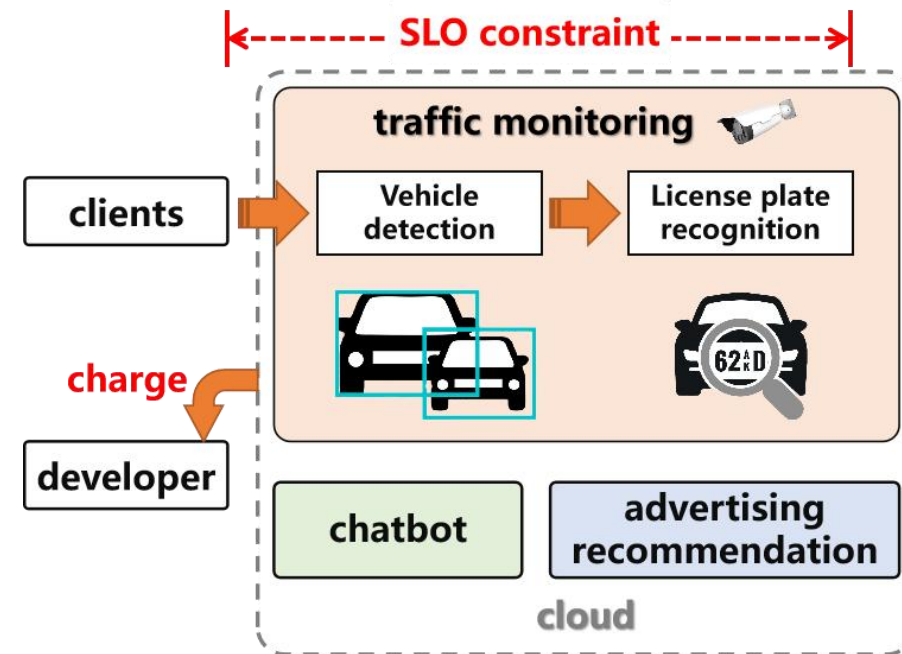
dynamic workloads



model cold starts



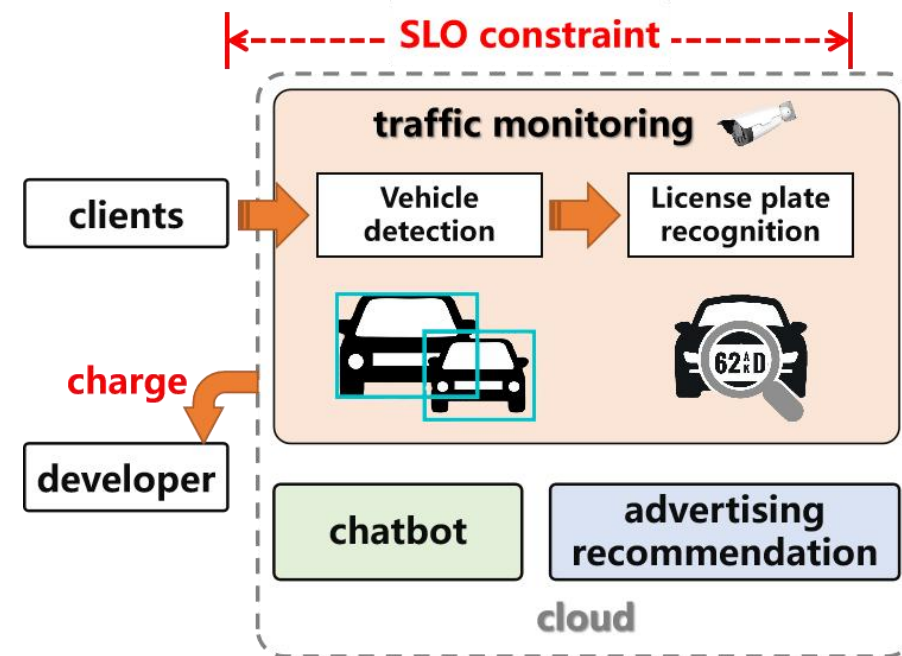
unpredictable events¹



How to guarantee P99 while minimizing cost ?

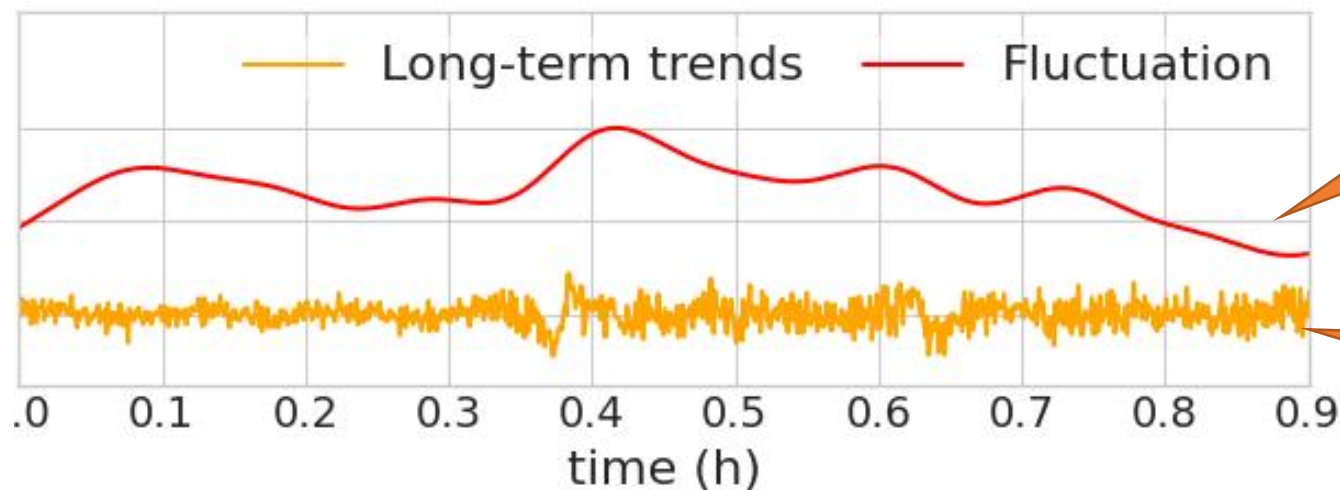
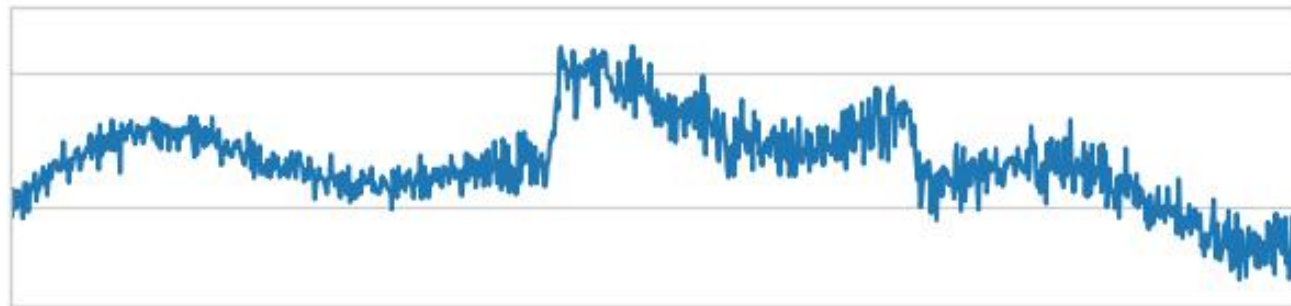
[1] Dean, Jeffrey, and Luiz André Barroso. "The tail at scale." *Communications of the ACM* 56.2 (2013): 74-80.

- Challenge
 - Dynamic workloads
 - Model cold starts
 - Unpredictable events
 - **Guarantee P99 & minimizing cost**
 - accurate scaling
- **workload estimation**



A: How to estimate workload accurately?

- Workload decomposition



existing solutions

- provisioning delay
- inaccurate prediction
- slow start

prediction

fluctuation

A: How to estimate workload accurately?

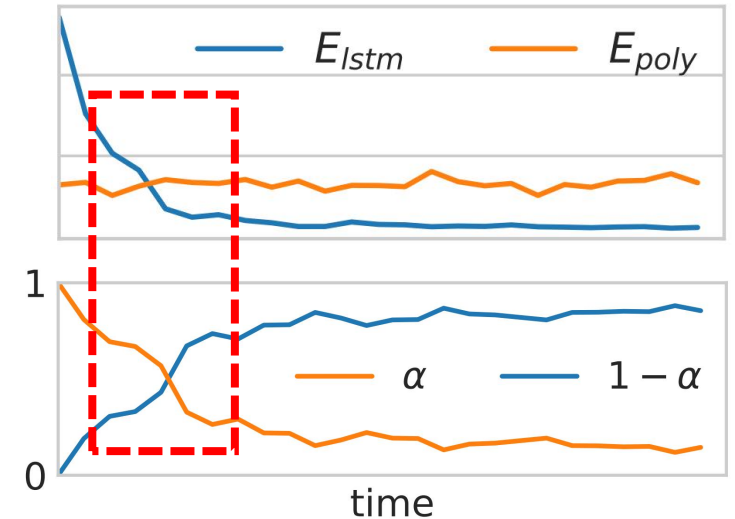
- **Dynamic Joint Prediction**

- Polynomial Regression + **Online Learning LSTM**

$$\tilde{R}_{t+\lambda}^{pred} = \alpha \tilde{R}_{t+\lambda}^{poly} + (1 - \alpha) \tilde{R}_{t+\lambda}^{lstm}$$

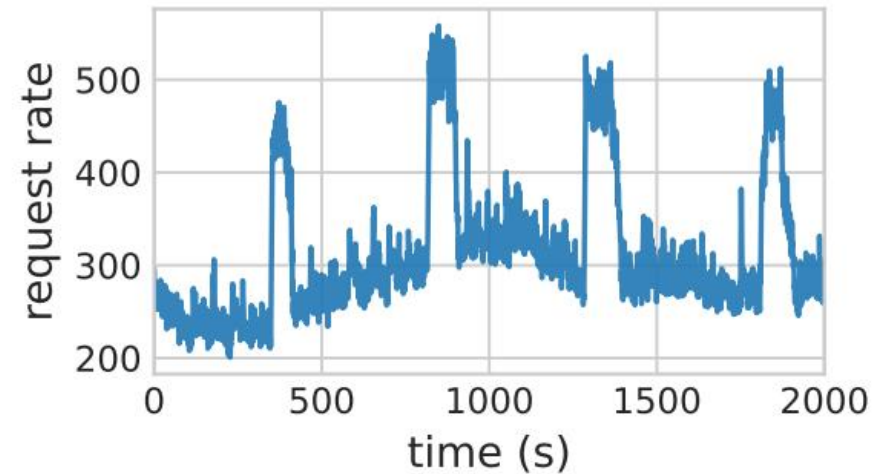
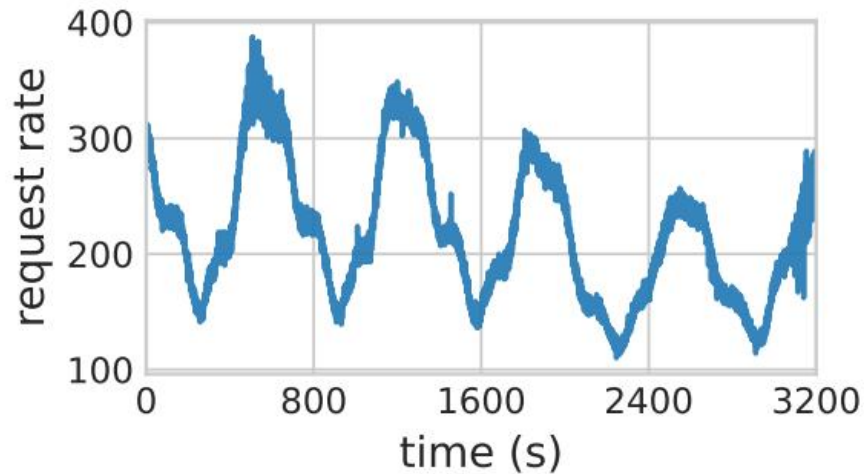
$$\alpha = \frac{E_{poly}}{E_{poly} + E_{lstm}}$$

weight update



A: How to estimate workload accurately?

Fluctuation is non-negligible and varying between workloads.



A: How to estimate workload accurately?

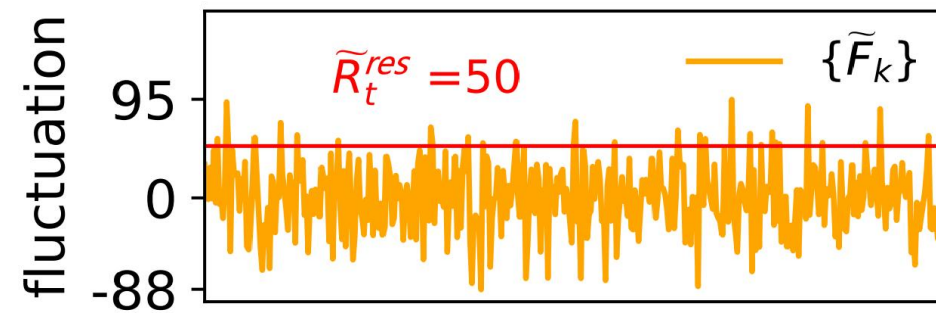
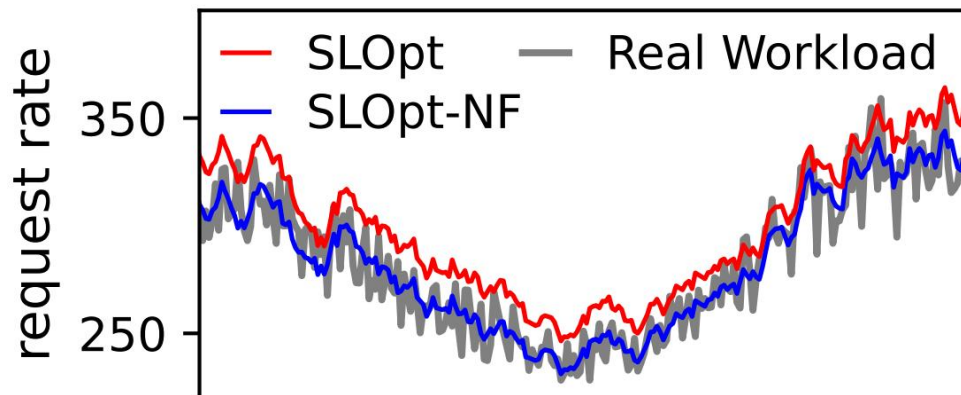
- **Fluctuation History**

- 95th percentile of workload fluctuation

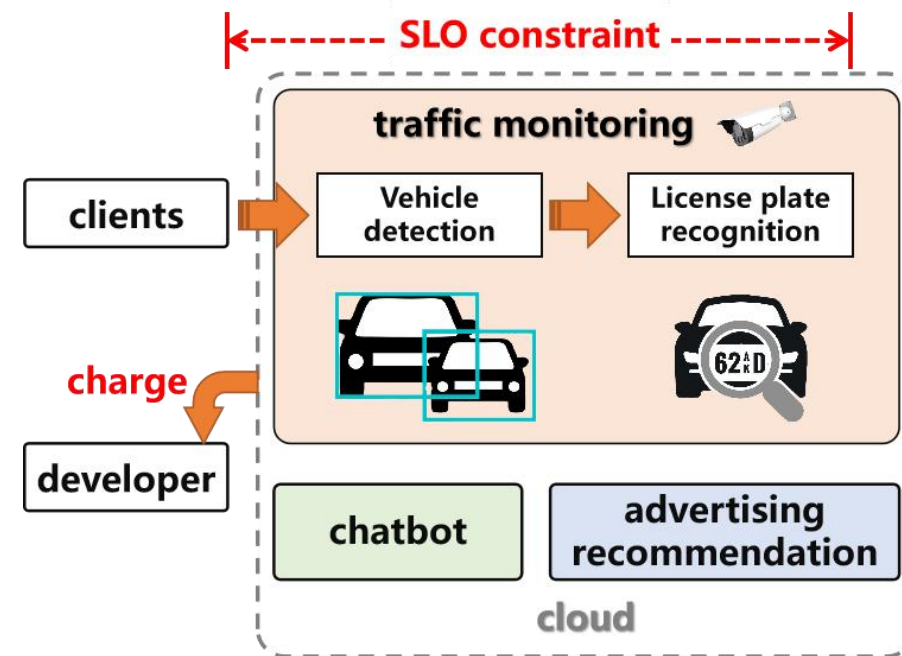
$$\tilde{R}_t^{res} = \{\tilde{F}_k\}^{P95}$$

$$\{\tilde{F}_k = \tilde{R}_k^{real} - \tilde{R}_k^{pred} | k = 1, 2, \dots\}$$

workload fluctuations



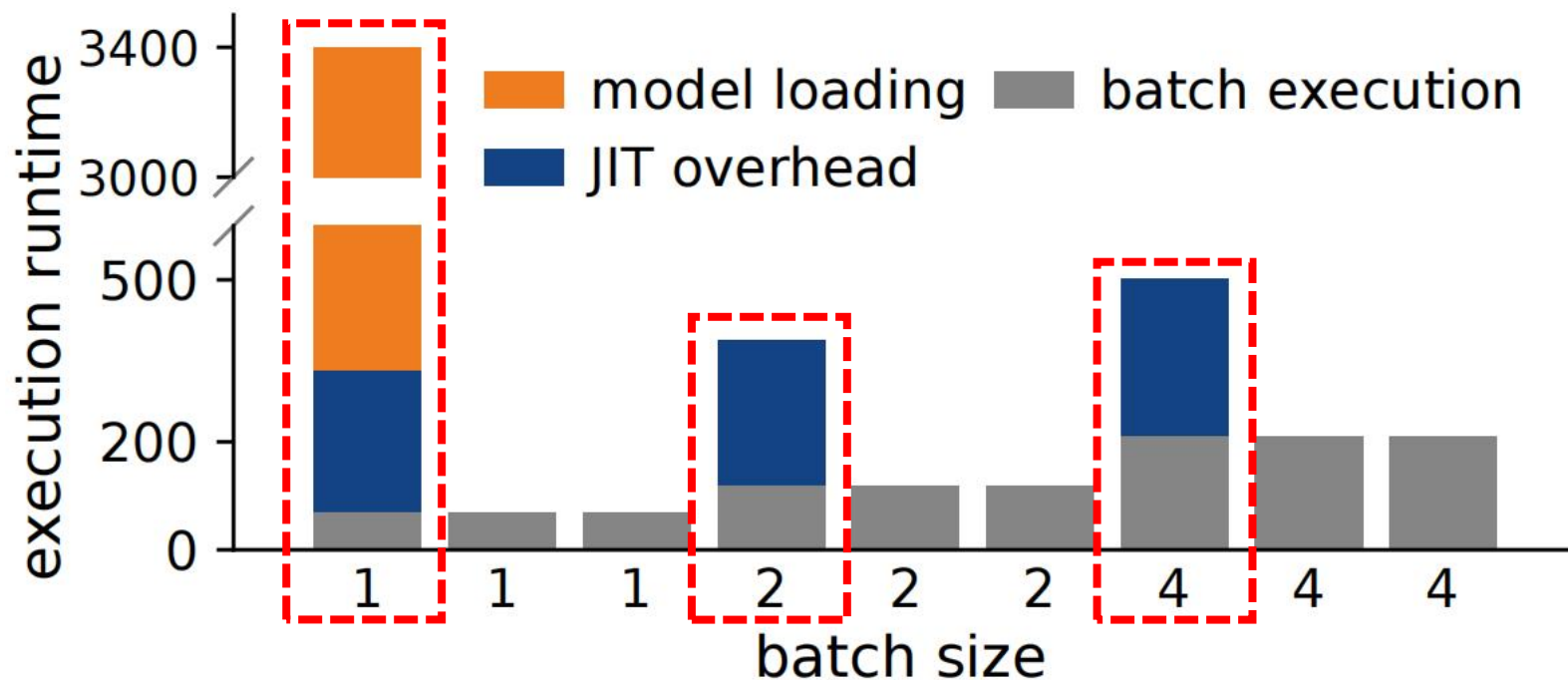
- Challenge
 - Dynamic workloads
 - Model cold starts
 - Unpredictable events
- **Guarantee P99 & minimizing cost**
 - accurate scaling
 - workload estimation
 - eliminate queueing
 - transient scheduling



B: How to mitigate model cold starts?

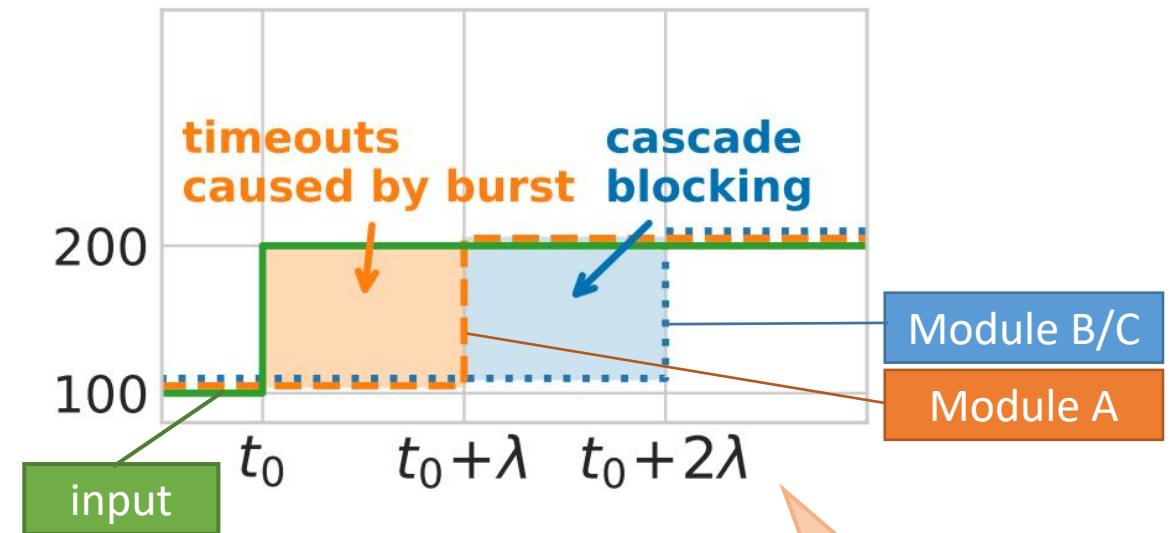
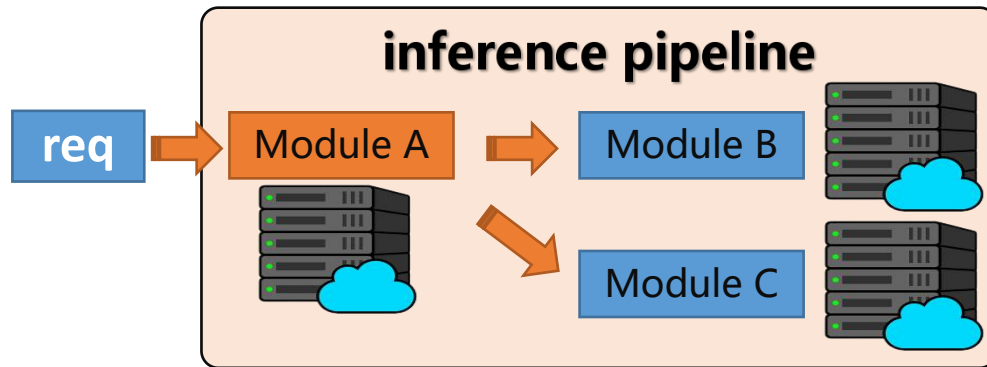
- model cold start

- model loading overhead
- JIT overhead



B: How to mitigate model cold starts?

- Model loading overhead in inference pipeline
- Inference pipeline suffers from **Cascade Blocking**.



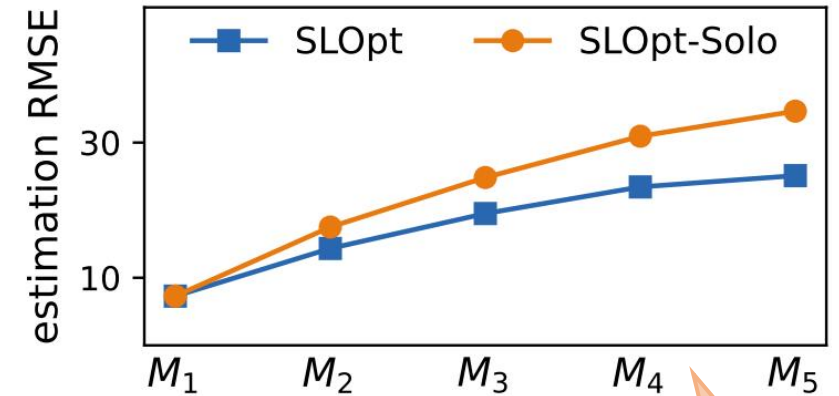
**throughput mismatch
caused by Cascade Blocking**

- Cascade Scaling:

$$\tilde{R}_{t+\lambda}^{cas}(M) = \tilde{S}_{t+\lambda}(M-1) \times \tilde{R}_{t+\lambda}(M-1)$$

$$\tilde{R}_{t+\lambda} = \max\{\tilde{R}_{t+\lambda}^{pred}, \tilde{R}_{t+\lambda}^{cas}\} + \tilde{R}_t^{res}$$

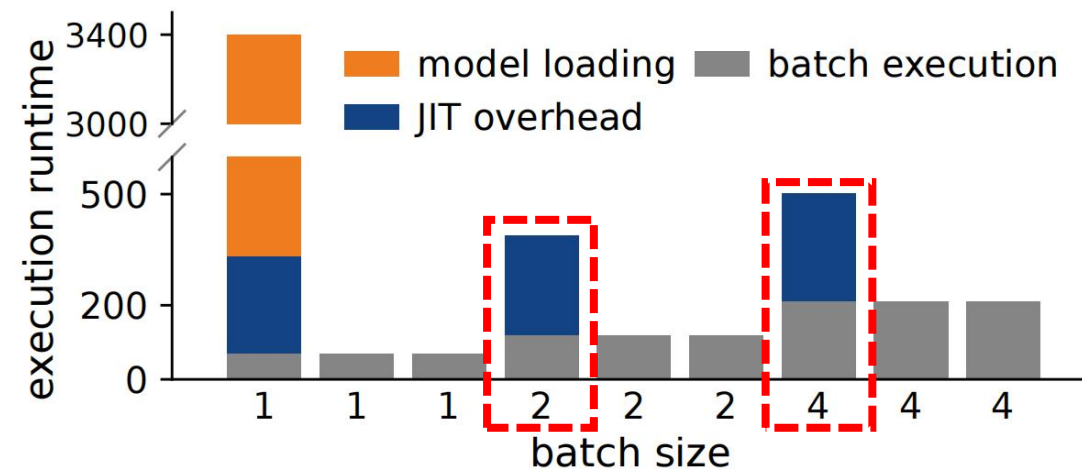
	Existing method			Cascade scaling		
	M_1	M_2	M_3	M_1	M_2	M_3
t_0	100	200	800	100	200	800
t_1	<u>150</u>	200	800	150	300	1200
$t_1+\lambda$	150	<u>300</u>	800	150	300	1200
$t_1+2\lambda$	150	300	<u>1200</u>	150	300	1200



reduce estimation
RMSE by up to 38%

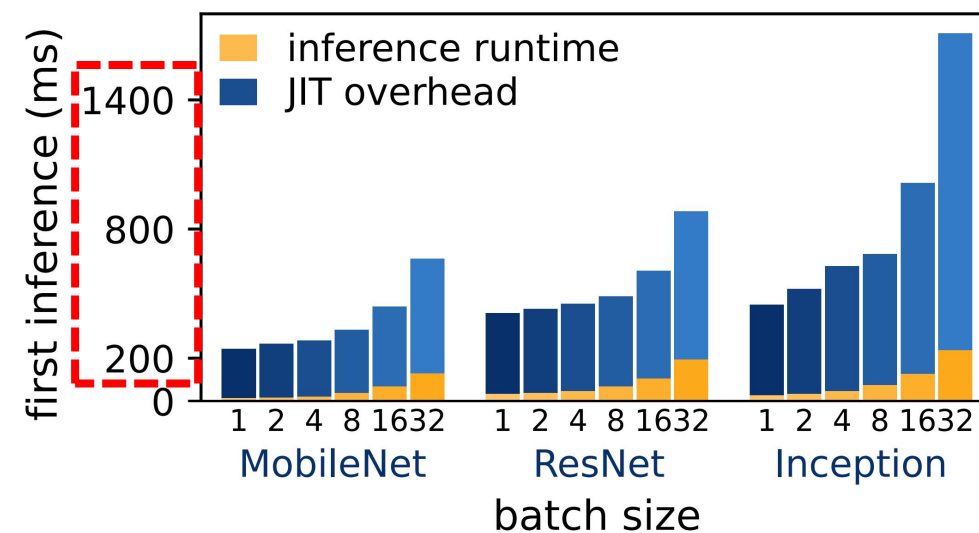
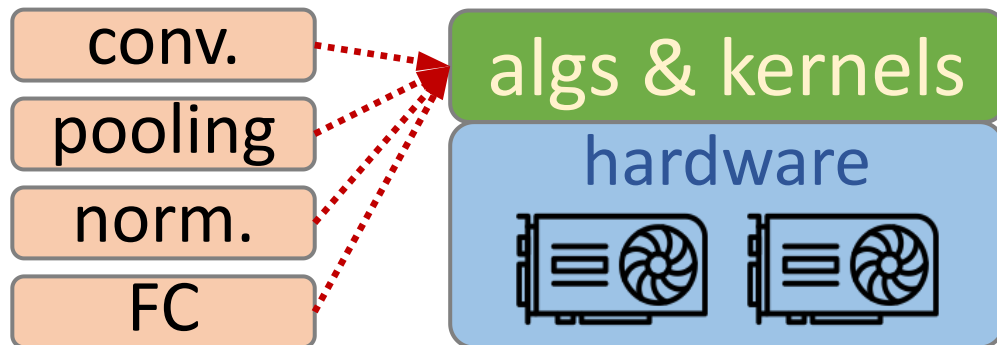
B: How to mitigate model cold starts?

- model cold start
 - model loading overhead
 - **JIT overhead (autotune)**



B: How to mitigate model cold starts?

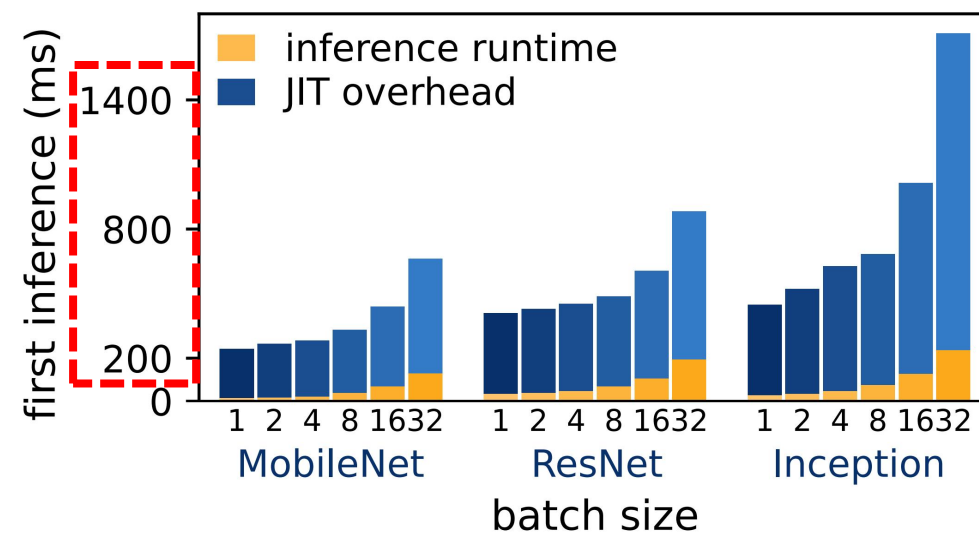
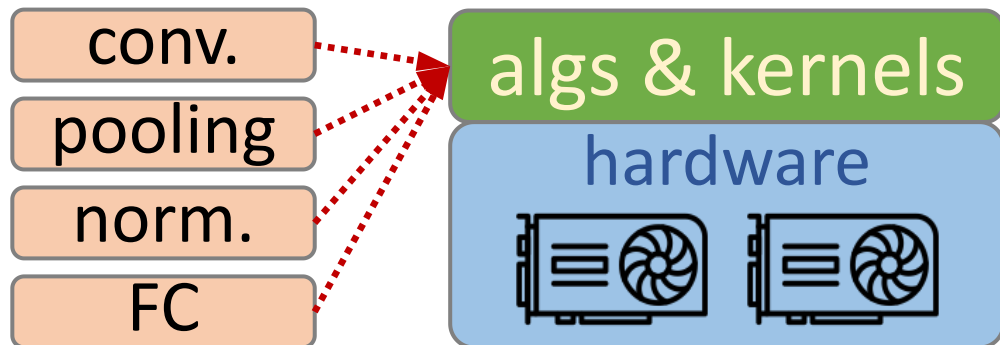
- model cold start
 - **JIT overhead (autotune)**



- Convolution -> alg {Direct Convolution, FFT Convolution, ...}
- Direct Convolution -> kernels {convolution_kernel, convolution_forward_kernel, ...}

B: How to mitigate model cold starts?

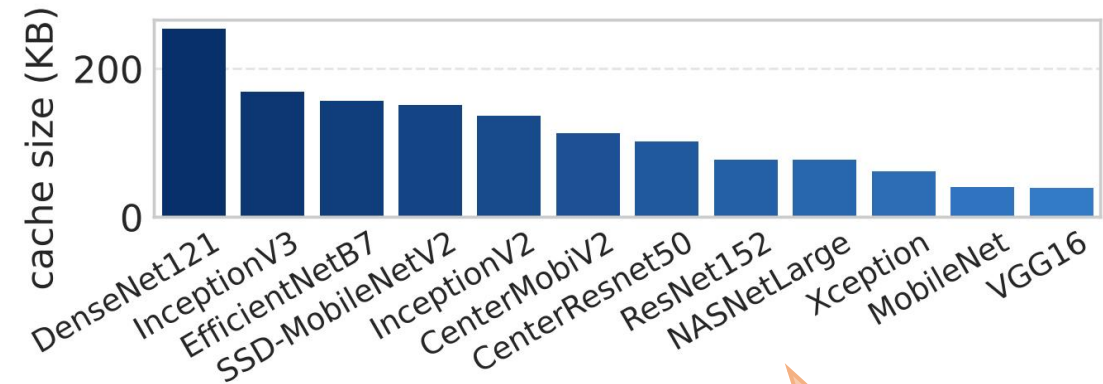
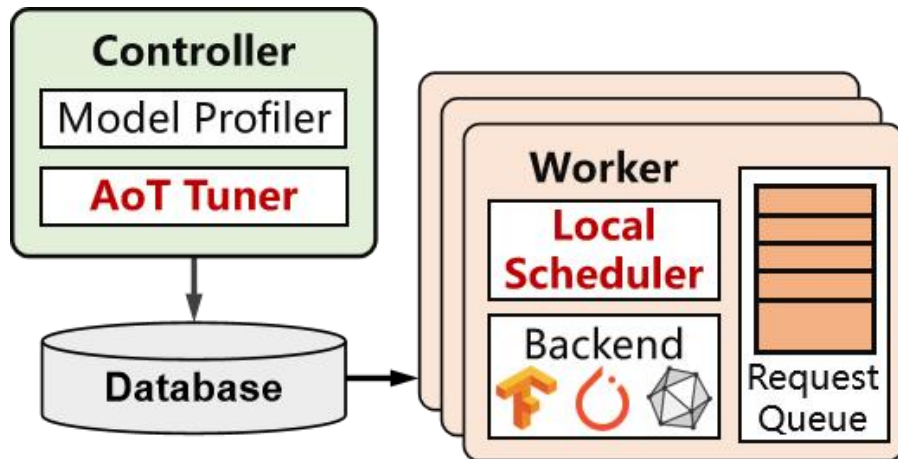
- model cold start
 - JIT overhead (autotune)



Mappings is fixed for a given batch size and hardware.

• Ahead-of-Time(AoT) Tuner

- Cache the mappings in advance through AoT Tuning.
- Load the mappings when the batch size changed.

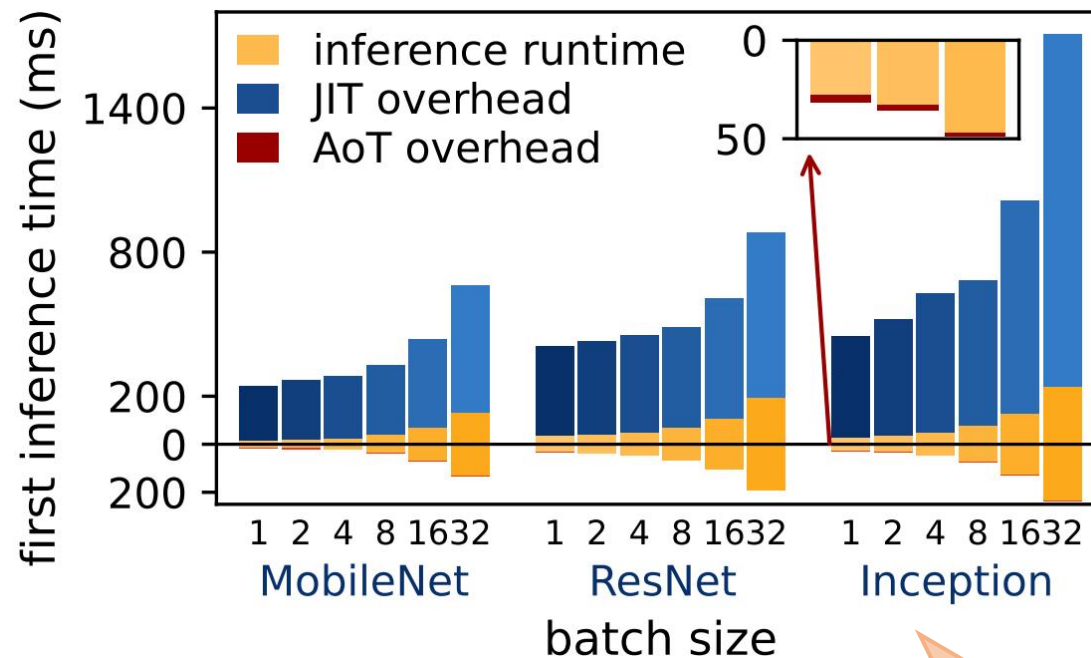
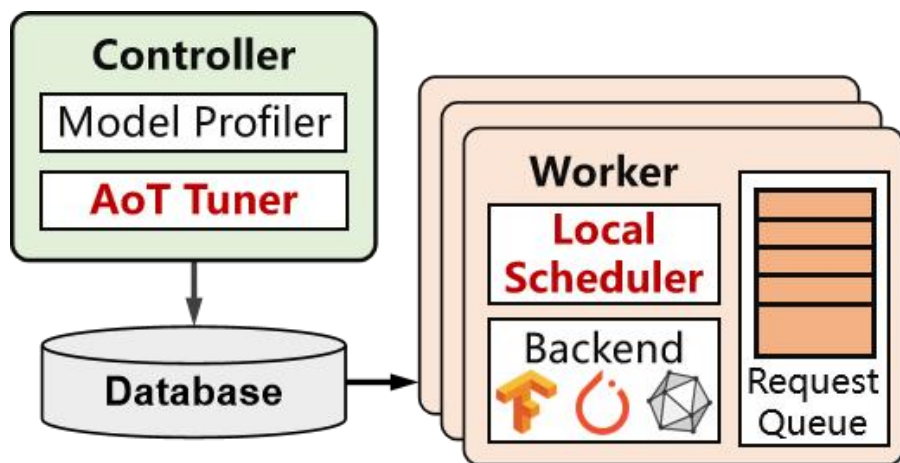


cache size 40-250KB!

B: How to mitigate model cold starts?

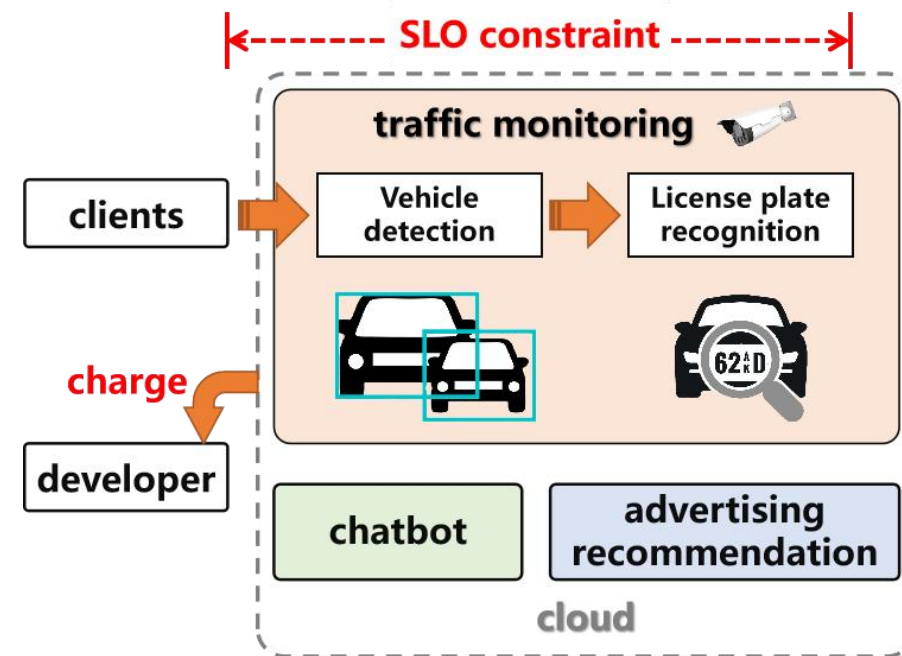
• Ahead-of-Time(AoT) Tuner

- Cache the mappings in advance through AoT Tuning.
- Load the mappings when the batch size changed.



reduce JIT overhead by >99%

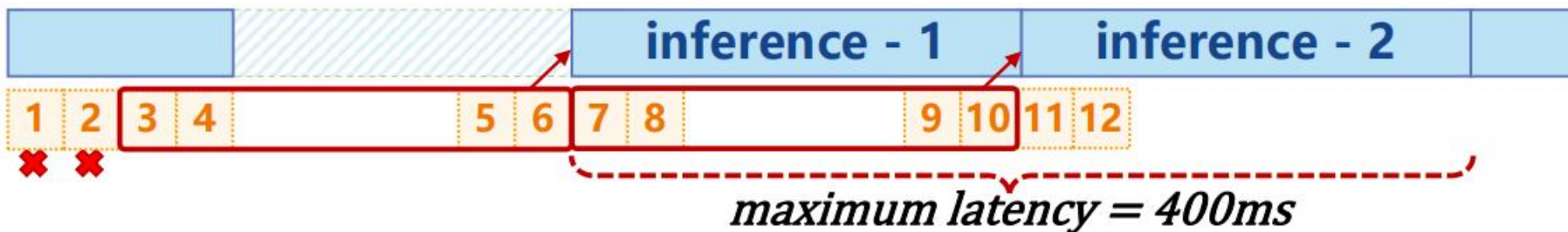
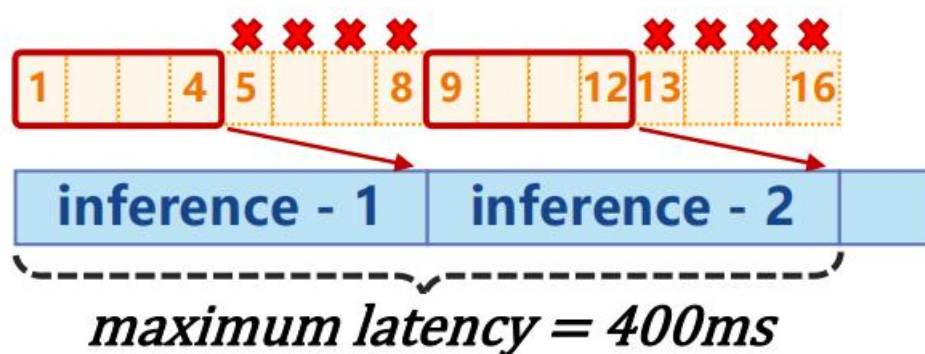
- Challenge
 - Dynamic workloads
 - Model cold starts
 - **Unpredictable events**
- **Guarantee P99 & minimizing cost**
 - accurate scaling
 - **workload estimation**
 - eliminate queueing
 - **transient scheduling**



C: How to drop request properly?

- Existing method

- Early drop, Lazy drop
- latency $\rightarrow$ SLO & reduce throughput

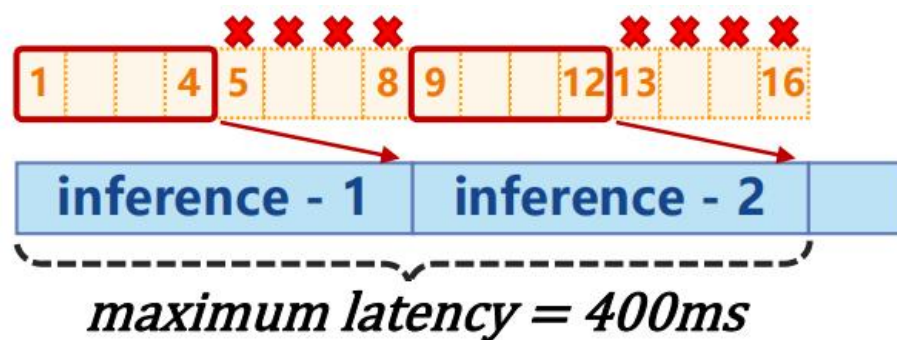


C: How to drop request properly?

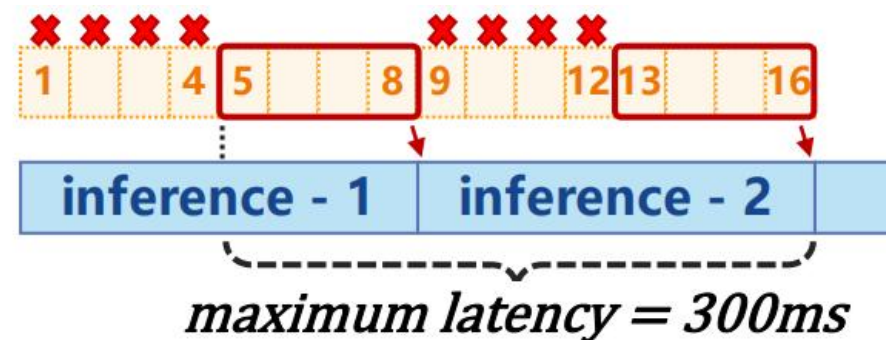
- **Adaptive Batch Drop**

- Inference the latest batch.

keep a larger gap between maximum latency and SLO



Early Drop



Adaptive Batch Drop

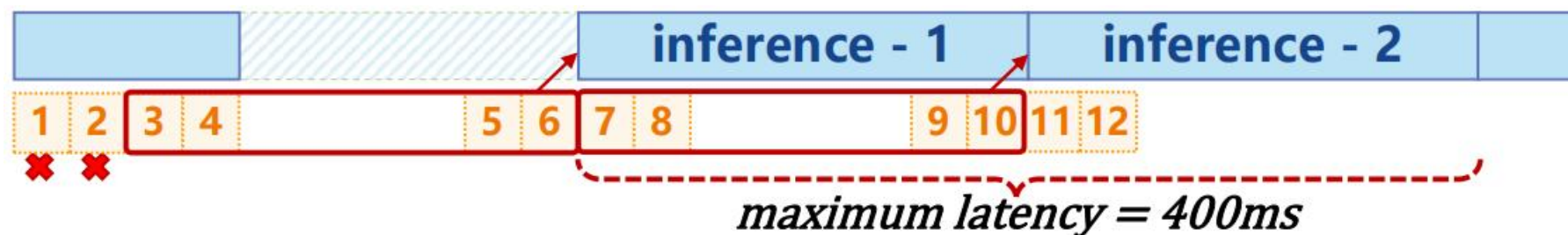
C: How to drop request properly?

• Adaptive Batch Drop

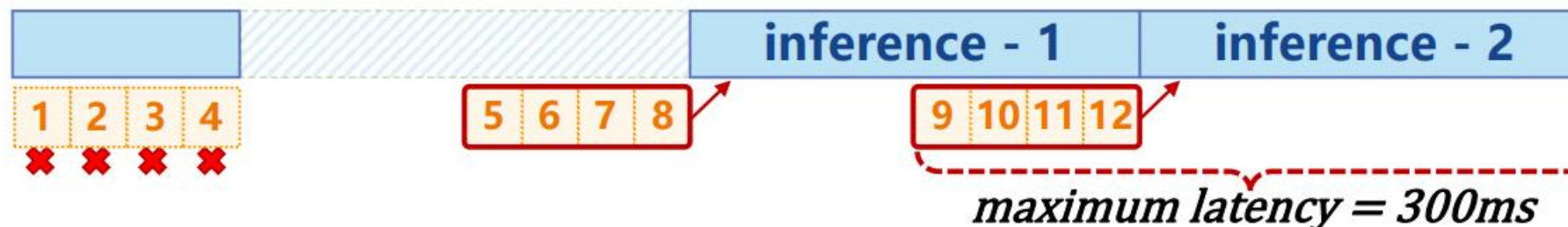
- Inference the latest batch.
- Drop the entire batch if any request can not satisfy SLO.

guarantee remaining batches
can be executed on plan

Early
Drop



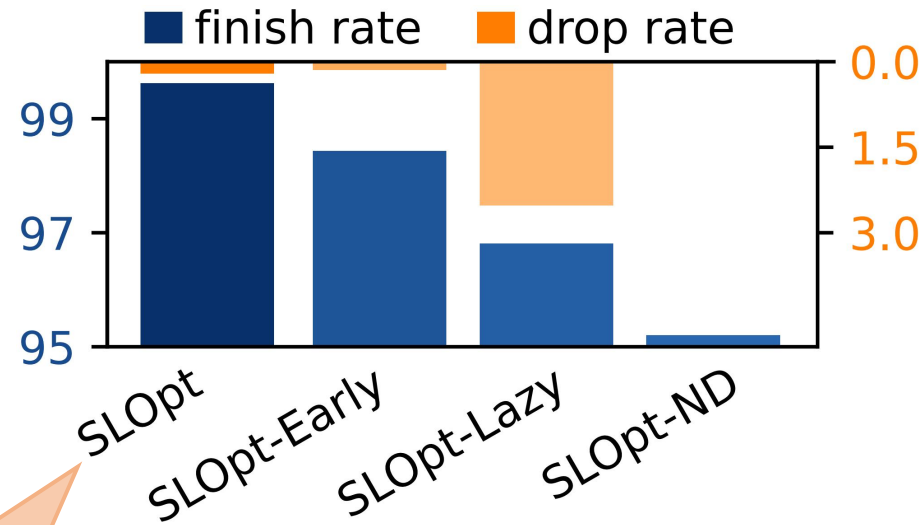
Adaptive
Batch Drop



C: How to **drop request** properly?

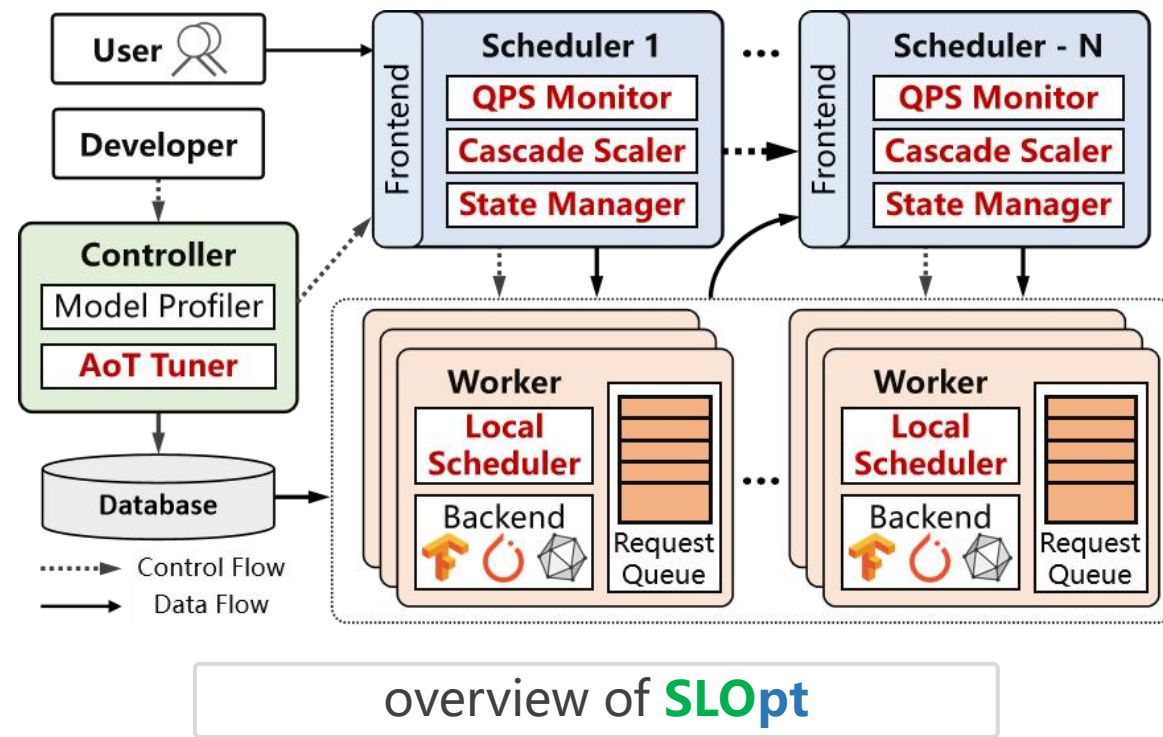
- **Adaptive Batch Drop**

- Inference the latest batch.
- Drop the entire batch if any request can not satisfy SLO.



**drops 0.8% more requests than Early Drop,
but has 1.19% higher finish rate.**

- **Accurate workload estimation**
 - DJP workload prediction
 - Fluctuation history
- **Novel transient scheduling policy**
 - Cascade Scaling
 - AoT Tuner
 - Adaptive Batch Drop
- **Fully implemented system**
 - ~7k lines of Python & C++ codes

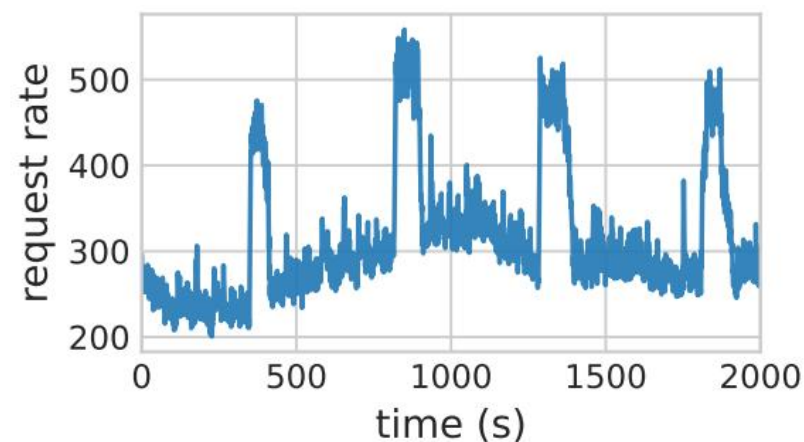
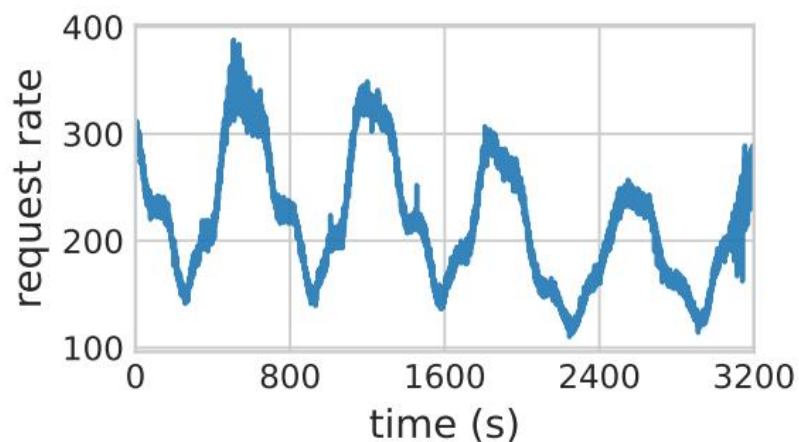


- **Baseline**

- Nexus¹ & InferLine²

- **Workload**

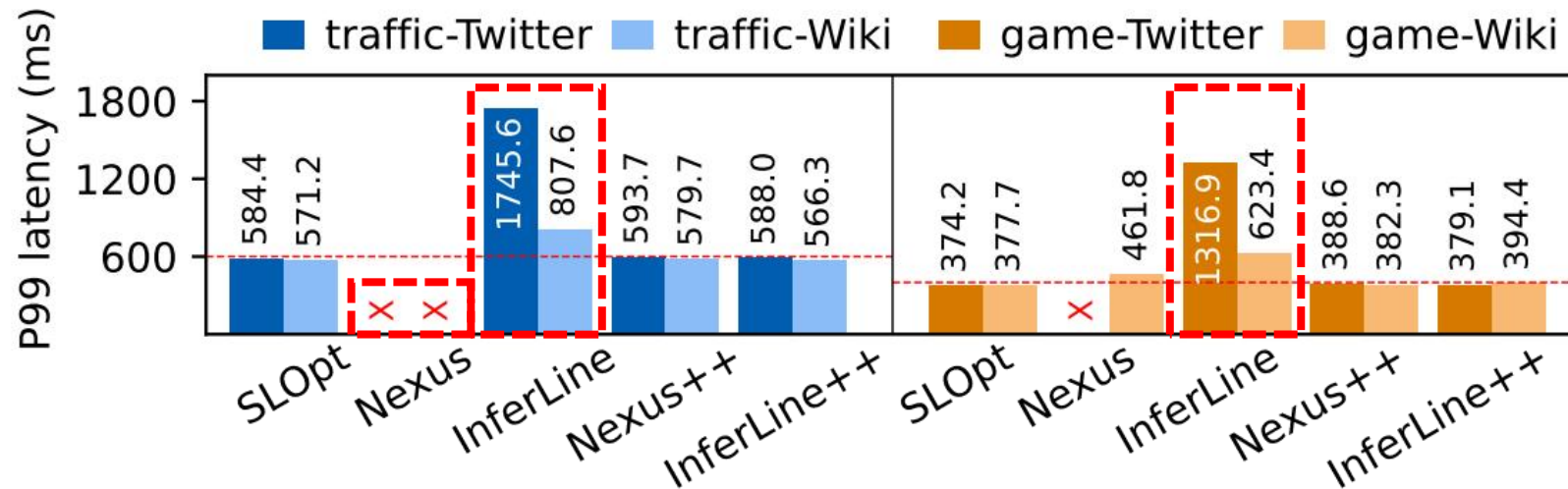
- Trace: Twitter & Wikipedia
- Application: Game video analysis & Traffic monitor



[1] Shen, Haichen, et al. "Nexus: A GPU cluster engine for accelerating DNN-based video analysis." *SOSP*. 2019.

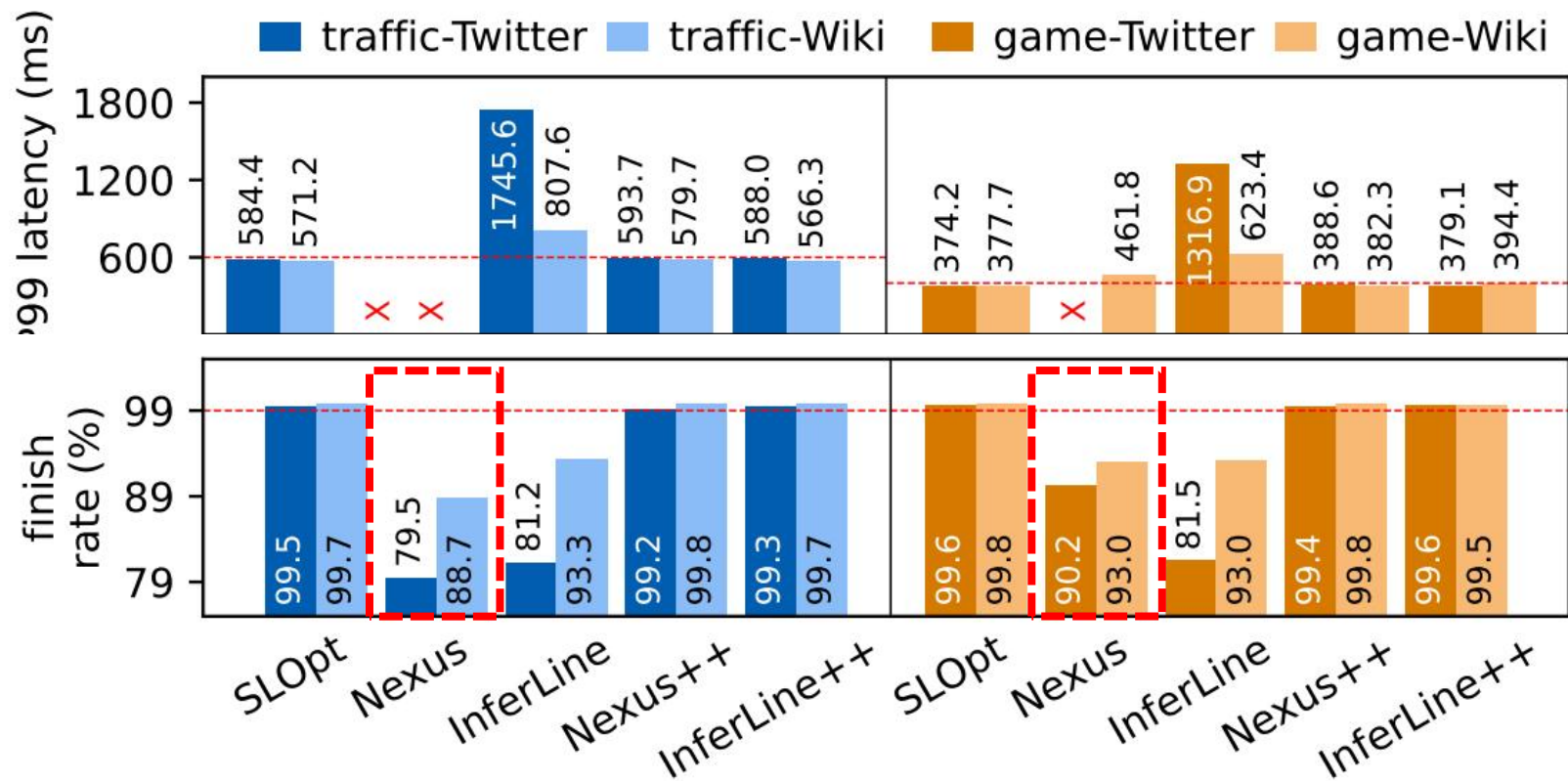
[2] Crankshaw, Daniel, et al. "InferLine: latency-aware provisioning and scaling for prediction serving pipelines." *SoCC*. 2020.

Comparison results: P99



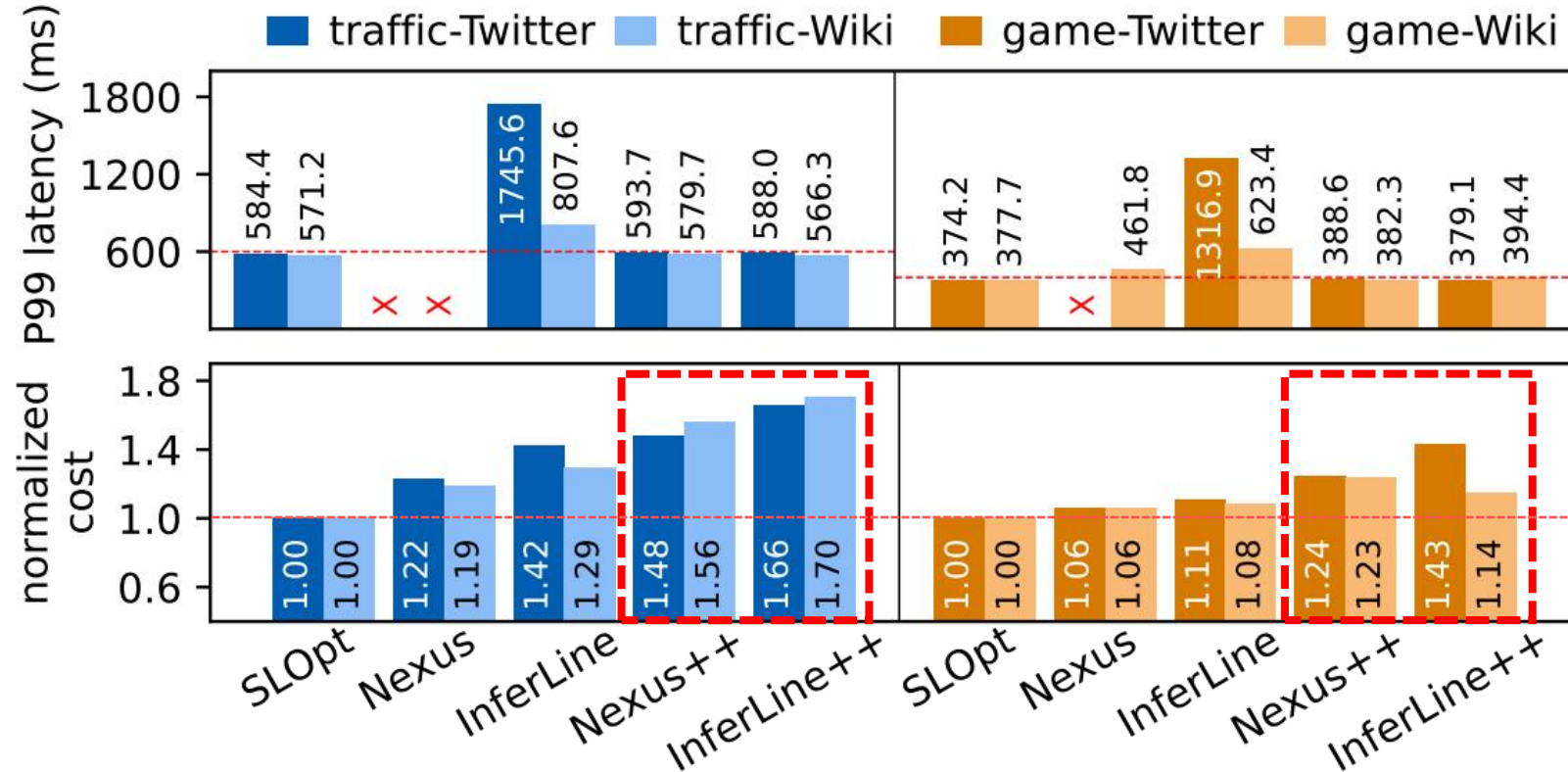
P99 of **InferLine** is 1.4 - 2.5x of **SLOpt**'s
Nexus drop more than 1% requests in most workloads

Comparison results: P99 & finish rate



Finish rate of **Nexus** is 6.7% - 19.8% lower than **SLOpt**'s

Comparison results: **serving cost**



To guarantee the P99 latency:

- **Nexus** needs 23.5% - 55.5% extra cost
- **InferLine** needs 14.4% - 70.3% extra cost

- **Accurate workload estimation**
 - Workload decomposition
 - **Dynamic Joint Prediction + Fluctuation History**
- **Novel transient scheduling policy**
 - Eliminate request queueing
 - **Cascade Scaling + AoT Tuner + Adaptive Batch Drop**
- **Evaluation**
 - Reduces the **P99 latency** by **1.4-2.5 times** and cost by 5.3%-29.7%.
 - Reduces the **cost** by up to **70.3%** under the same quality of service.



SLOpt: Resource Scaling for DNN Workload with Tail Latency Constraint

Zhixin Zhao, Yitao Hu, Guotao Yang, Yingqin Chen, Wenxin Li, Wenyu Qu

May 2023